BZ205 Konu Anlatımı

Listeler ve Bağlantılı Listeler

Hocanın ders videoları, slaytları ve sınav soruları temel alınmıştır

27 Ekim 2026

4 Listeler ve Bağlantılı Listeler

Konu üç derste anlatıldı: **8 Sıralama 5** (son yarım saat: soyut veri tipi, liste işlemleri, dizi ile gerçekleştirme), **9 Liste 1** (bağlantılı listenin yapısı, `Node`/`List` sınıfları, `InsertNode`, `FindNode`) ve **10 Liste 2** (`DeleteNode`, `DisplayList`, yıkıcı, `main`, liste çeşitleri, polinom; yaklaşık 42. dakikadan sonrası yığın konusudur). Aşağıdaki video dakikaları bu derslere aittir.

4.1 Veri yapısı ve soyut veri tipi

Hocaya göre veri yapısı aslında bir **depolama ünitesidir**. Bir veri yapısı tanımlarken iki şey belirlenir:

1. **Nesneler kümesi:** ne depolanacak? (`int` tam sayı, `float` ondalık, `char` karakter, öğrenci kaydı…)
2. **İşlemler kümesi:** bu veriler üzerinde hangi işlemler yapılacak? `int` için toplama, çıkarma, çarpma, bölme. `char` için "a + b" toplama değil, yan yana koyma ("ab") olabilir; "−" de iki kelimenin harf farkı olabilir.

Soyut veri tipi (SVT) = soyut veri yapısı (SVY): dildeki temel tipleri kullanarak *kendi* oluşturduğumuz, davranışını da kendimizin programladığı yeni tiptir. Dizi, string, char, byte gibi yapılar eskiden soyuttu; artık kütüphanede hazır geldikleri için *temel veri yapısı* sayılıyor.

! Hoca derste vurguladı [video 8 Sıralama 5 30:52]

"Soyut veri yapısıyla soyut veri tipi aynı şey." Slaytların bir kısmında SVY, bir kısmında SVT geçer.

- **Benzetme** [video 8 Sıralama 5 43:01]: inşaat mühendisliğinde kum, çimento ve tutkal temel malzemedir; birleşince beton olur. Bitler birleşip byte'ı, byte'lar birleşip string ve dizileri oluşturur.
- **Soyutlama:** `x + y` yazarken toplamanın nasıl yapıldığını düşünmeyiz; biri yazıp kütüphaneye koymuş. SVT de bunu sağlar.
- **class – struct farkı:** class içinde değişkenler *ve* fonksiyonlar tanımlanabilir (fonksiyonu sadece sınıf içindekiler kullanır); struct'ta fonksiyonlar dışarıda tanımlanır ve başkası da kullanabilir (**risk**). Bu yüzden SVT C++'ta `class` ile tanımlanır; işlemler üye fonksiyon olur.
- **SVT örnekleri:** küme (birleşim, kesişim, boyut, tersini alma), kuyruk (oluştur, ekle, incele, sil, yok et). *Adları aynı olsa da iki SVY aynı değildir*; arkadaki fonksiyonlar farklı olabilir.

- **Avantajlar:** *modülerlik* (hoca: *"bir şirkete gittiğimizde bir iki ay kütüphanelerimizi incele derler"* [video 8 Sıralama 5 39:18]; debug kolaylaşır, birden çok kişi aynı anda çalışır), *bilgi gizleme*, *hızlı prototipleme*.
- **Dezavantajı, verimlilik kaybı:** tarih-saat yapısı yazılmış, siz yalnız tarihi kullanıyorsunuz ama saat kısmı da arkada yüklü [video 8 Sıralama 5 41:22].

4.2 Liste SVT ve temel işlemler

Liste $A_1, A_2, \ldots, A_N$ biçiminde art arda dizilmiş elemanlardır: N uzunluk, A_1 ilk, A_N son eleman; $N = 0$ ise **boş liste**. A_i, A_{i+1}'den önce gelir ve A_{i-1}'i takip eder. **Sıralı olması gerekmez.** Slayt örneği: **34, 12, 52, 16, 12** (öğrenci numarası gibi düşünün).

İşlem	Sonuç	Açıklama
`printList`	–	listeyi ekrana yaz (sınıf listesinin çıktısı)
`makeEmpty`	–	listeyi boşalt (*"dönem bitti, sınıf listesini boşaltmam lazım"*)
`find(52)`	3	*konumu* döner; sıralı olmadığı için baştan tek tek bakılır; yoksa 0/NULL
`insert(3, x)`	34, 12, 52, **x**, 16, 12	3. elemandan *sonra* ekle: trende 3. vagondan sonra makaslayıp araya vagon koymak
`remove(52)`	34, 12, x, 16, 12	bul ve çıkar; öndeki ve arkadaki vagonu birbirine bağla
`findKth(5)`	12	baştan 5. eleman; k'yı kullanıcı verir (*"kura çektik, 60. kişi ödül alacak"* [video 8 Sıralama 5 48:06])

Ders varsayımı: her veriden bir tane vardır (birden fazla olsaydı fonksiyonlar değiştirilmeliydi).

! Hoca derste vurguladı Sayma neden 1'den başlıyor? [video 8 Sıralama 5 45:49]

Öğrenci sorusu: *"Hocam neden sıfırdan başlamıyor?"* Cevap: bağlantılı listede *"head'i sıfır kabul ettiği için birden başlar."* Düğümler 1, 2, 3… diye sayılır; `InsertNode(0, x)` "head'den hemen sonra" demektir.

SVT iki şekilde gerçekleştirilir: **dizi tabanlı** ve **bağlantılı liste tabanlı**.

4.3 Dizi ile gerçekleştirme

Bağlantılı liste, dizinin şu dört dezavantajını ortadan kaldırmak için önerilmiştir:

1. **Sabit boyut:** boyut derleme zamanında verilir; 10'luk diziye 11. eleman eklenemez (hata verir ya da eklenmez).
2. **Blok hafıza:** 6'lık dizi için hafızada *art arda* 6 boş hücre gerekir; bir sürü 5'lik boşluk olsa da dizi oraya sığmaz [video 8 Sıralama 5 28:05].
3. **Tek tip veri:** dizi `int` ise her hücresi `int`'tir.
4. **Hücrede tek veri:** öğrencinin no, ad ve soyadı için 3 ayrı dizi gerekir; "1. hücreler aynı öğrencinin" eşleşmesini yalnız programcı bilir.

Ekleme/silme: sona ekleme veya sondan silme sorunsuzdur. *Başa* ekleme yer açmak için tüm elemanları bir hücre aşağı kaydırır; baştan silme boş hücre kalmasın diye tüm elemanları yukarı kaydırır. Ortalamada listenin yarısı yer değiştirir.

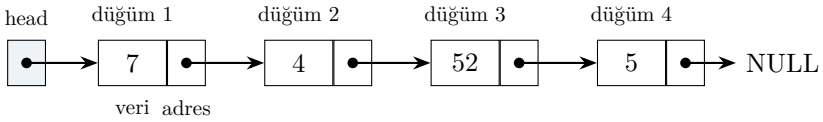
İşlem	Dizide	Neden
`printList`	$O(n)$	n elemanın hepsi yazılır
`find`	$O(n)$	aranan en sonda olabilir
`findKth`	$O(c)$, sabit	`dizi[k]` ile doğrudan gidilir
`insert`	$O(n)$	başa eklemede $n-1$ eleman kayar
`remove`	$O(n)$	baştan silmede kalan elemanlar yukarı kayar

! Hoca derste vurguladı [video 8 Sıralama 5 55:21 ve 56:16]

Maksimum boyut önceden bilinmelidir; aksi hâlde yer israfı olur: *"kontenjanla 40 kişi"* açılan seçmeli derse 20 kişi yazılırsa 20 hücre boşa durur. `findKth` karmaşıklığını hoca önce farklı söyledi, bir öğrenci sabit olduğunu belirtti; hoca *"doğru söylüyorsun"* dedi.

4.4 Bağlantılı listenin kavramsal yapısı

Elemanların hafızada art arda durması gerekmez: hafızanın neresine düşerse düşsün, her elemandan *sonrakine* bir bağlantı atılır. Hoca bunu **tren** ile anlatır:



- **head (baş, lider) = lokomotif:** listenin başlangıcını gösterir, **içinde veri yoktur** (*"lokomotifte verinin olmaması gibi"* [video 9 Liste 1 13:01]).
- **Düğüm (node) = vagon:** veri + *sonraki düğümün adresi* (işaretçi). 1. düğümde 2'nin adresi, 2'de 3'ün adresi vardır. Veri kısmında *birden fazla ve farklı tipte* veri olabilir (no, ad, soyad, okul…).
- **Son düğümün işaretçisi NULL'dır.** Tahtada topraklama sembolü (veya boş küme işareti) ile çizilir: *"topraklama sembolü programlamada NULL'a denk gelir"* [video 9 Liste 1 06:56]. Aramada NULL'ı görünce durulur; 3 elemanlı listeye `insert(5, x)` denirse NULL görülür ve kullanıcıya hata verilir.
- **Diziden farkı:** dizide işaretçi yoktur, başlangıcı bilmek yeter (`int` 4 byte ise ilk 4 byte 1. hücredir).
- **Dezavantajı, işaretçi için ek yer** [video 9 Liste 1 10:40]: veri 1 MB, işaretçi 16 KB ise ihmal edilir; ama veri küçükse işaretçiler çok yer kaplar ve tek avantaj veriyi dağınık tutabilmek kalır.
- **Bağlantı koparsa** [video 9 Liste 1 12:09]: vagonlar arasındaki bağlantıyı kaybetmek gibi, *"bağlantılı listenin arasındaki verileri de kaybederiz"*; hafızada hiçbir yere bağlı olmayan serseri veri kalır.

4.5 Dizi ile bağlantılı liste karşılaştırması

	Dizi	Bağlantılı liste
Boyut	*Statik*: sabit, derleme anında verilir	*Dinamik*: teoride sınırsız, pratikte bellek kadar
Bellek	Ardışık *blok* hafıza gerekir (6'lık dizi 5'lik boşluklara sığmaz)	Düğümler bellekte dağınık olabilir, bağlantılarla birleşir
Veri tipi	Tek tip, hücrede tek veri (öğrenci için 3 ayrı dizi: no, ad, soyad)	Düğümde birden çok/farklı tipte veri (no, ad, soyad birlikte)
Ek yer	Yok	Her düğümde işaretçi
Başa ekleme/silme	Tüm elemanlar kayar: $O(n)$	Yalnız işaretçiler değişir: $O(1)$
k. eleman (findKth)	Sabit: $O(1)$	Baştan gezilir: $O(n)$
Bulma, yazdırma	$O(n)$	$O(n)$

Bağlantılı listeyi kodlamak ve yönetmek diziden zordur, ama ekleme/silme kolay ve hızlıdır.

4.6 Bağlantılı liste gerçekleştirmesi (C++)

Önce kavramsal yapı kâğıt kalemle tasarlanır, sonra **iki sınıfla** programlanır: `Node` düğümü, `List` listeyi ve işlemlerini tanımlar.

```
class Node {
public:
    double data;      // veri (ogrenci icin no, ad, soyad olabilir)
    Node*  next;      // sonraki dugume isaretci
};
class List {
public:
    List(void) { head = NULL; }             // yapici: bos liste
    ~List(void);                            // yikici
    bool  IsEmpty() { return head == NULL; }
    Node* InsertNode(int index, double x);
    int   FindNode(double x);
    int   DeleteNode(double x);
    void  DisplayList(void);
private:
    Node* head;       // ilk dugumu gosterir; verisi yok (lokomotif)
};
```

- `data`: anlatım basit olsun diye tek bir `double`. İstenirse `int ogrNo;`, `string ad;`, `string soyad;`, `int bolumKodu;` gibi alanlarla genişletilir.
- `next`: sonraki eleman da bir `Node` olduğu için işaretçi *kendi cinsinden* bir adresi tutar: `Node*`.
- **public / private** [video 9 Liste 1 19:05]: public her yerden erişilir (`Node`'un `data` ve `next`'ine dışarıdan yazılabilir); private'a yalnız sınıfın kendi fonksiyonları erişir. `protected` ders kapsamında değil. `head` *private*'tır: listenin başlangıcını dışarıdan kimse değiştirememeli. head de bir `Node*`'dır ama verisi yoktur.
- **Yapıcı (constructor)** [video 9 Liste 1 23:19]: sınıfla aynı isimde, girdisi `void`; değişken tanımlandığı anda *otomatik* çalışır. Hoca: `int x;` satırını gören derleyici hafızada yer ayırıp "x burası" der; `List ogrenciListesi;` satırını görünce de yapıcı çalışır, head'i oluşturur, NULL atar ve bekler. Kalıtım detayları kapsam dışı.
- **Yıkıcı (destructor)** [video 9 Liste 1 26:03]: *"tilda işareti olumsuzluk ekidir."* Kapsam (scope, ör. `main`'in kapanış parantezi) bitince otomatik çalışır ve hafızayı bırakır; yoksa orası bloklu kalır. Gövdesi uzun olduğu için sınıfın dışında yazılır.

- `IsEmpty` [video 9 Liste 1 29:10]: `==` mantıksal karşılaştırmadır; head NULL'ı gösteriyorsa true. En önemli kullanımı *silme*: liste boşsa silinecek bir şey yoktur.

Fonksiyon	Girdi	Dönüş
`InsertNode(index, x)`	index'inci düğümden *sonra* x'i ekle	eklenen düğümün *adresi* (`Node*`); başarısızsa NULL
`FindNode(x)`	x'i ara	*konumu* (3, 5, 7…); yoksa 0
`DeleteNode(x)`	x'i bul ve sil	sildiği düğümün konumu; yoksa 0
`DisplayList()`	yok	yok (`void`)

Remove ile delete aynı şeydir; karışmasın diye hepsinin sonuna "Node" eklenmiştir. Parametreler `double`'dır çünkü `data` double; birden fazla veri olsaydı `FindNode(ogrNo)`, `DeleteNode(ogrNo)` gibi yazılırdı.

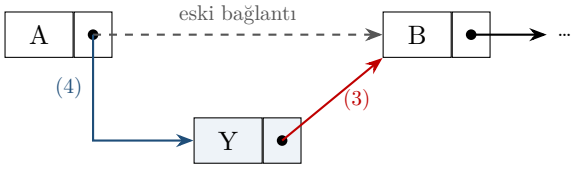
! Hoca derste vurguladı [video 9 Liste 1 35:18]

"Bir çözüm anlatacağız ama farklı çözümler de var literatürde. Haberiniz olsun." Tek doğru illa bu değil; ama ders kapsamında bu kod üzerinden gidilir.

InsertNode – belirtilen konumdan sonra ekleme

Mantık: index'inci düğüm bulunur, oradaki bağlantı koparılıp araya "kaynak yapılır". Hoca bunu **dört adımda** anlatır [video 9 Liste 1 36:38] (önceki düğüm A, sonraki B, yeni düğüm Y):

1. index'inci elemanı **bul**; bulunamazsa hata ver ve çık.
2. Yeni düğüm için **yer aç** (`new Node`) ve x'i içine kopyala (hafıza doluysa düğüm oluşturulamaz).
3. **Yeni düğüm sonrakini göstersin:** A'nın next'indeki B adresi Y'nin next'ine kopyalanır.
4. **Önceki düğüm yeni düğümü göstersin:** Y'nin adresi A'nın next'ine yazılır.



Dört durum, kodda **iki grup**: (1) boş listeye ekleme, (2) en öne ekleme ⇒ ikisi de *head'den hemen sonra* (`index == 0`); (3) sona ekleme (yeni düğüm artık NULL'ı gösterir), (4) ortaya ekleme ⇒ `else` kolu.

! Hoca derste vurguladı [video 9 Liste 1 42:11]

"Head değişmiyor arkadaşlar. Dikkat edin. Head hiçbir zaman değişmeyecek." Lokomotif aynı kalır, vagonlar değişir. head değişkeni yapıcıda bir kez oluşturulur; başa eklemede değişen şey yalnız *head'in tuttuğu adrestir* (`head = newNode`). Veriye ihtiyaç olunca head'in gösterdiği düğümden başlanır.

```
Node* List::InsertNode(int index, double x) {
    if (index < 0) return NULL;                        // 1. hata kontrolu
    int currIndex = 1;
    Node* currNode = head;
    while (currNode && index > currIndex) {            // ADIM 1: bul
        currNode = currNode->next;
        currIndex++;
```

```
    }
    if (index > 0 && currNode == NULL) return NULL;   // 2. hata kontrolu
    Node* newNode = new Node;                          // ADIM 2
    newNode->data = x;
    if (index == 0) {                                  // head'den hemen sonra
        newNode->next = head;                          // ADIM 3
        head = newNode;                                // ADIM 4
    } else {                                           // ortaya / sona
        newNode->next = currNode->next;                // ADIM 3
        currNode->next = newNode;                      // ADIM 4
    }
    return newNode;
}
```

- `List::` fonksiyonun List sınıfına ait olduğunu söyler (sınıf dışında yazıldığı için).
- **1. hata kontrolü:** head 0. eleman olduğu için negatif indeksten sonra ekleme olamaz; NULL "eklenemedi" demektir.
- `currIndex = 1`, `currNode = head`: "şu an 1 numaralı düğümdeyim".
- **while'da iki kriter** `&&` ile bağlıdır, ikisi de doğru olmalı: `currNode` NULL değil (liste bitmedi) *ve* `index > currIndex` (aranan yere gelinmedi). Gövde bir sonraki düğüme zıplar ve indeksi artırır.
- **2. hata kontrolü:** index > 0 olduğu hâlde currNode NULL olduysa öyle bir konum yoktur (ör. 4 düğümlü listeye `InsertNode(10, 25)`).
- `index == 0` kolunda head'in eski bağlantısı üzerine yazıldığı için kendiliğinden kopar.
- **İşaretçi nedir?** Hoca [video 9 Liste 1 60:15]: *"Bizim bir evimiz var, evin adresi var; her yere evi taşımaya gerek yok, o adrese posta gelir."*

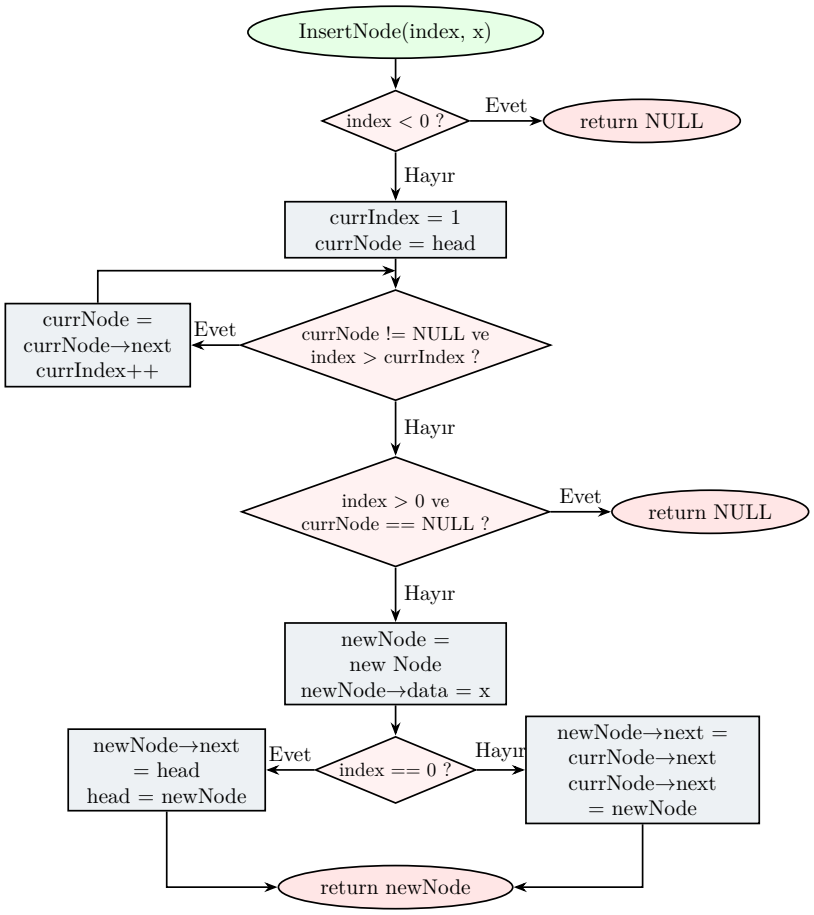
Bağlantı sırası

Araya eklemede önce `newNode->next = currNode->next`, *sonra* `currNode->next = newNode` yapılır. Ters sırada yapılırsa B'nin adresi kaybolur ve listenin geri kalanı serseri veri olur.

Hocanın izlemesi: 4 düğümlü listede `InsertNode(3, 25)` [video 9 Liste 1 50:20 civarı]

Adım	currNode	currIndex	`currNode` $\neq$ NULL	$3 >$ currIndex	Sonuç
ilk değer	A1	1	✓	$3 > 1$ ✓	döngü, zıpla
1. tur sonu	A2	2	✓	$3 > 2$ ✓	döngü, zıpla
2. tur sonu	A3	3	✓	$3 > 3$ ×	çık: 3. düğüm bulundu

2. hata kontrolü geçilir, 25'li düğüm oluşturulur ve `else` kolu 3. ile 4. düğüm arasına ekler.



Yürütme zamanı (çıkmış soru: *"bağlantılı liste ekleme akış diyagramı, karmaşıklığı"*). En kötü durum sona eklemedir (index $= n$): while başlığı n kez kontrol edilir, gövde $n-1$ kez çalışır. Hoca iki kriterli koşulu (`&&`) **2 işlem** sayar [video 5 Sıralama 2 33:49]:

Satır	En kötü (sona ekleme)	En iyi (`index` $= 0$)
`if (index < 0)`	1	1
`currIndex = 1; currNode = head`	2	2
`while (… && …)`	$2n$	2
gövde (2 satır)	$2(n-1)$	0
`if (index > 0 && …)`	2	2
`new Node; data = x`	2	2
`if (index == 0)`	1	1
iki bağlantı ataması	2	2
`return newNode`	1	1
Toplam	$\mathbf{T(n) = 4n + 9 \Rightarrow O(n)}$	$\mathbf{13 \Rightarrow\ (1)}$

`&&` 1 işlem sayılırsa $3n + 8$ ve 11 çıkar (çözüm dosyasındaki tablo); her iki sayımda da sonuç $O(n)$ ve $\Omega(1)$'dir. Dereceler farklı olduğu için Θ verilmez. Dizide başa ekleme $O(n)$ idi; bağlantılı listede sabittir.

FindNode – arama

Hoca: `InsertNode`'daki while'ı *modifiye ediyoruz*; ikinci kriter `currNode->data != x` olur.

```
int List::FindNode(double x) {
    Node* currNode = head;
    int currIndex = 1;
```

```
    while (currNode && currNode->data != x) {
        currNode = currNode->next;
        currIndex++;
    }
    if (currNode) return currIndex;            // bulundu: konumu
    return 0;                                  // bulunamadi
}
```

Hocanın izlemesi: 5 → 10 → 52 → 3 listesinde `FindNode(52)` [video 9 Liste 1 65:22]

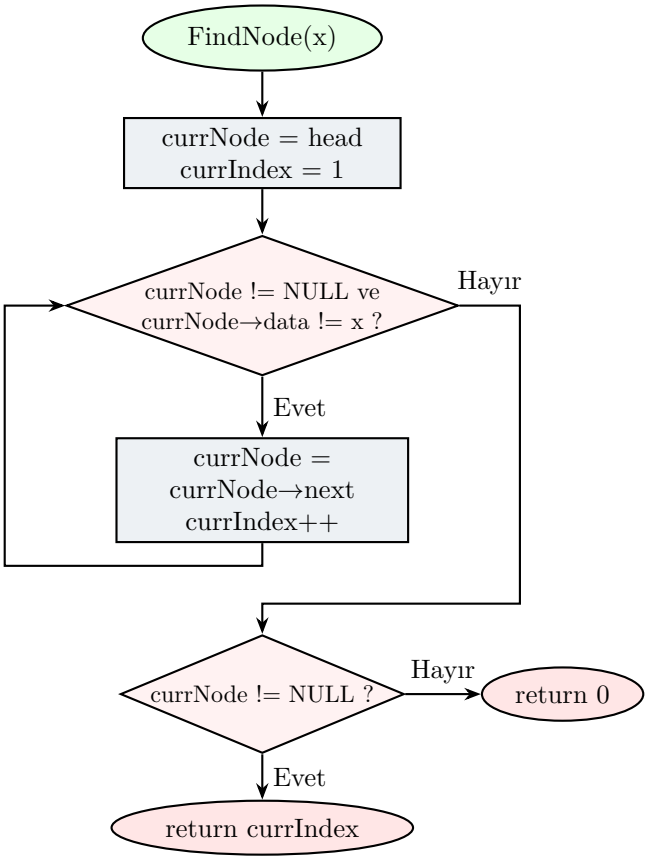
currNode	data	currIndex	data = 52?	Sonuç
1. düğüm	5	1	hayır	döngü, zıpla
2. düğüm	10	2	hayır	döngü, zıpla
3. düğüm	52	3	**evet**	çık, `currNode` dolu ⇒ **3** döner

currNode NULL ise sona gelinmiş, bulunamamıştır ve 0 döner.

! Hoca derste vurguladı [video 9 Liste 1 65:03]

x'ten birden fazla varsa? *"İlk bulduğu X'i bize döndürecek. İlk bulduğundan sonra başka eleman varsa bu kod çalışmaz."* Hepsini bulmak için döngü listenin sonuna kadar çalıştırılmalıdır.

Akış diyagramı: FindNode (listede arama)



Farz edin ki sorularında listede öğrenci aramanın kalıbıdır.

DeleteNode – silme

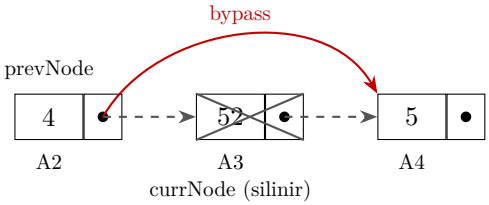
InsertNode ile hemen hemen aynı mantık: (1) silinecek düğümü bul, (2) düğümü sil, (3) silinen düğümün *öncesini* ve *sonrasını* bağla (trenden vagon çıkarmak). İki durum vardır: *ilk düğümü* (head'den hemen sonrakini) silmek ve *ortadan/sondan* silmek. Önceki düğümün next'i değişeceği için **`prevNode`** (previous node) tutulur; FindNode'a göre while'a eklenen tek satır budur.

```
int List::DeleteNode(double x) {
    Node* prevNode = NULL;
    Node* currNode = head;
    int currIndex = 1;
    while (currNode && currNode->data != x) {
        prevNode = currNode;                    // oncekini kaybetme
        currNode = currNode->next;
        currIndex++;
    }
    if (currNode) {                             // bulundu
        if (prevNode) {                         // ortadan / sondan sil
            prevNode->next = currNode->next;
            delete currNode;
        } else {                                // ilk dugumu sil
            head = currNode->next;
            delete currNode;
        }
        return currIndex;
    }
    return 0;                                   // bulunamadi
}
```

Hocanın izlemesi: 7 → 4 → 52 → 5 listesinde `DeleteNode(52)` (PN = prevNode, CN = currNode, CI = currIndex)

Adım	PN	CN	CI	CN.data = 52?
ilk değer	NULL	A1	1	7, hayır ⇒ döngü
1. tur sonu	A1	A2	2	4, hayır ⇒ döngü
2. tur sonu	A2	A3	3	52, **evet** ⇒ çık

CN dolu (bulundu), PN dolu (ortadan silme): A3'ün next'indeki adres A2'nin next'ine kopyalanır, A3 silinir, **3** döner.



! Hoca derste vurguladı [video 10 Liste 2 14:37 ve 16:42]

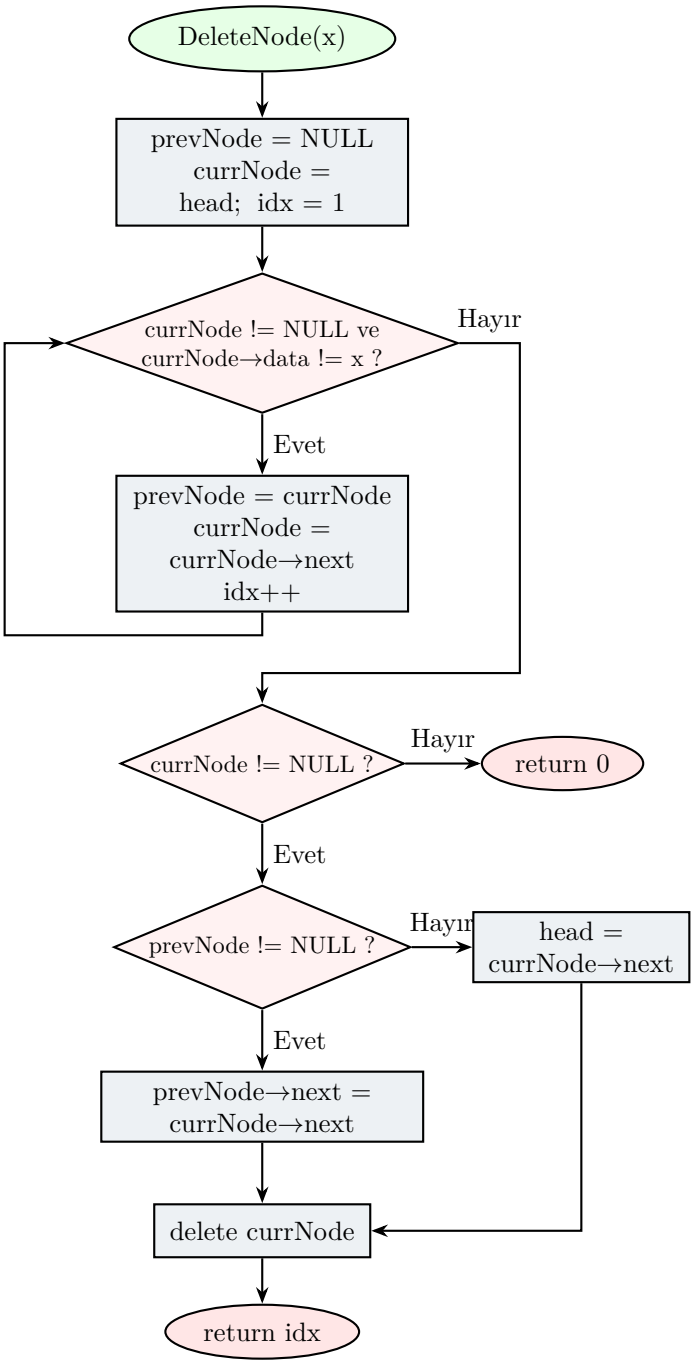
"Bypass yapıyorum. Yani aslında bir bypass ameliyatı gerçekleştirmiş oluyorum."
Öğrenci: `delete currNode` yazmasak olmaz mı, zaten erişilemiyor? Cevap: düğüm *"garbage değer oluyor"*; başka bir program "hafıza boşsa yaz" dediğinde orayı dolu görür. Java'da garbage collector var; biz risk almayıp kendimiz sileriz.

İlk düğümü silme (`else`): döngüye hiç girilmediği için prevNode NULL kalmıştır. Bu durumda `head = currNode->next` ataması ile head'den bypass yapılır, sonra düğüm silinir. (Derste hoca bir an "currNode'da NULL var" dedi; kodda NULL olan `prevNode`'dur.)

Tek elemanlı liste: Yunus'un sorusu [video 10 Liste 2 17:08]

head → [52] → NULL, `DeleteNode(52)`: PN = NULL, CN = A1, CI = 1. Veri 52 olduğu için döngüye girilmez. `if (currNode)` sağlanır, `if (prevNode)` sağlanmaz ⇒ `else: head = currNode->next` = NULL, düğüm silinir. Geriye yalnız NULL'ı gösteren head kalır; fonksiyon **1** döndürür.

Akış diyagramı: DeleteNode (listeden silme)



Akış diyagramında iki koldaki `delete currNode` tek kutuda birleştirilmiştir; davranış aynıdır.

DisplayList ve yıkıcı

```
void List::DisplayList() {
    int num = 0;                                // kac dugum yazildi (istege bagli)
    Node* currNode = head;
    while (currNode != NULL) {
        cout << currNode->data << endl;         // printf de olurdu
        currNode = currNode->next;
        num++;
    }
    cout << "Number of nodes in the list: " << num << endl;
}
List::~List(void) {
    Node* currNode = head, *nextNode = NULL;
    while (currNode != NULL) {
        nextNode = currNode->next;              // listenin ucunu tut
        delete currNode;
        currNode = nextNode;
    }
}
```

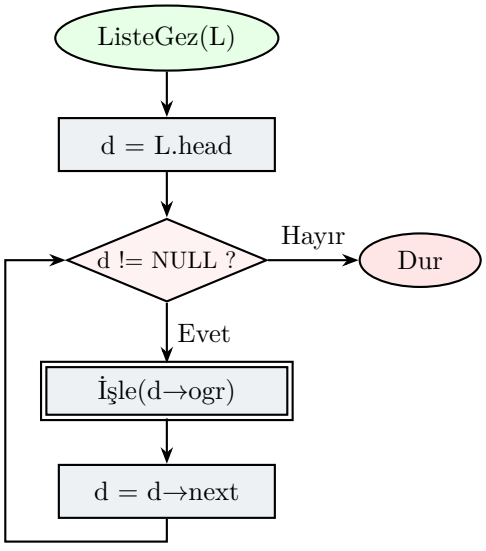
DisplayList (D1 → D2 → D3): CN = A1 → D1 yazılır, num = 1; A2 → D2, num = 2; A3 → D3, num = 3; CN = NULL olunca döngüden çıkılır ve düğüm sayısı yazılır. `endl` yüzünden veriler alt alta çıkar.

Yıkıcı 3 düğümde: CN = A1, NN = A2, A1 silinir, CN = A2; NN = A3, A2 silinir, CN = A3; NN = NULL, A3 silinir, CN = NULL ⇒ bütün liste baştan sona silinmiştir.

! Hoca derste vurguladı [video 10 Liste 2 26:26]

`nextNode` geçici değişkendir: *"listenin ucunu tutuyor, ucunu bırakmıyor."* Önce silip sonra next'e bakmaya kalkarsak silinmiş düğümün next'ine erişemeyiz.

Akış diyagramı: listeyi baştan sona gezme (DisplayList kalıbı)



Kullanım: `main`

```
int main(void) {
    List list;                      // yapici calisir: head = NULL
    list.InsertNode(0, 7.0);        // basarili
    list.InsertNode(1, 5.0);        // basarili
```

```
    list.InsertNode(-1, 5.0);      // basarisiz
    list.InsertNode(0, 6.0);       // basarili
    list.InsertNode(8, 4.0);       // basarisiz
    list.DisplayList();
    if (list.FindNode(5.0) > 0) cout << "5.0 found" << endl;
    else                        cout << "5.0 not found" << endl;
    if (list.FindNode(4.5) > 0) cout << "4.5 found" << endl;
    else                        cout << "4.5 not found" << endl;
    list.DeleteNode(7.0);
    list.DisplayList();
    return 0;                      // kapsam biter: yikici calisir
}
```

Komut	Sonuç	Liste
`List list;`	yapıcı: head oluşur, NULL atanır	(boş)
`InsertNode(0, 7.0)`	head'den sonra	7
`InsertNode(1, 5.0)`	1. düğümden sonra	7, 5
`InsertNode(-1, 5.0)`	**başarısız**: negatif indeks	7, 5
`InsertNode(0, 6.0)`	head'den sonra araya kaynak	6, 7, 5
`InsertNode(8, 4.0)`	**başarısız**: 3. düğümden sonra NULL görülür	6, 7, 5
`DisplayList()`	6 7 5, "Number of nodes in the list: 3"	
`FindNode(5.0)`	3 döner, $> 0 \Rightarrow$ "5.0 found"	
`FindNode(4.5)`	0 döner $\Rightarrow$ "4.5 not found"	
`DeleteNode(7.0)`	7 bypass edilip silinir	6, 5
`DisplayList()`	6 5, "Number of nodes in the list: 2"	
`return 0;`	kapsam biter, yıkıcı listeyi temizler	

! Hoca derste vurguladı [video 10 Liste 2 29:51]

Sınıf ve fonksiyon tanımları **`main`'in içinde yapılmaz**; ya `main`'den önce yazılır ya da bir kütüphaneye konup `#include` ile eklenir (öğrencilerin sık yaptığı hata). Fonksiyonlar nokta ile çağrılır; değişken adı serbesttir (`List veriYapilariOgrenciListesi;` de olur).

▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

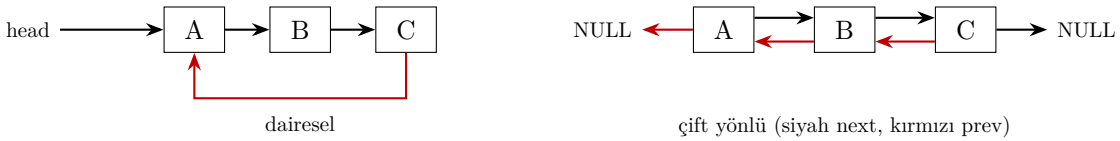
Zaman ve alan karmaşıklığı

İşlem	En iyi	En kötü	Ek alan	Not
`InsertNode` (konum k)	$\Omega(1)$	$O(n)$	$O(1)$	en iyi başa ekleme; en kötü sona ekleme, $T(n) = 4n + 9$ (`&&` 1 sayılırsa $3n + 8$)
`FindNode`	$\Omega(1)$	$O(n)$	$O(1)$	en iyi aranan ilk düğümde; en kötü sonda veya yok
`DeleteNode`	$\Omega(1)$	$O(n)$	$O(1)$	silinecek düğüm önce aranır; bağlantıyı değiştirmek sabit
`DisplayList`	$\Theta(n)$	$\Theta(n)$	$O(1)$	her düğüm bir kez yazılır
`findKth`(k)	$\Omega(1)$	$O(n)$	$O(1)$	dizide $O(1)$; listede baştan k düğüm gezilir

4.7 Liste çeşitleri

- **Dairesel (çevrimsel) bağlantılı liste** [video 10 Liste 2 35:30]: son düğüm NULL yerine *ilk düğümü* gösterir; head ile içeri girilip içeride dönülür. Gezmeyi ne zaman bitireceğiz? NULL hiç gelmez; `currNode`'un yeniden başa (head'in gösterdiği düğüme) gelip gelmediği kontrol edilir.
- **Çift yönlü bağlantılı liste** [video 10 Liste 2 36:24]: her düğümde `next` ve `prev` vardır; sağa ve sola gidilebilir, **iki NULL** bulunur (ilk düğümün prev'i, son düğümün next'i). Bir elemanı

bulduktan sonra bir öncekine gitmek için tek yönlü listede *yine baştan başlamak* gerekir; çift yönlüde tek adım geri gidilir. Silmede önceki düğümü ayrıca tutmaya gerek kalmaz.



dairesel çift yönlü (siyah next, kırmızı prev)

4.8 Uygulama: polinom SVT

$f(x) = \sum a_i x^i$. **Dizi ile** a_i katsayısı i'nci hücreye konur; katsayılar art arda doluysa sorun yoktur. Ama

$$P_1(x) = 10x^{1000} + 5x^{14} + 1, \qquad P_2(x) = 3x^{1990} - 2x^{1492} + 11x + 5$$

için P_1 1001 hücre ister ve yalnız 3'ü dolar (hoca derste "997 tane boş" dedi [video 10 Liste 2 39:51]; tam sayı 998). Bu iki polinomu çarparken zamanın çoğu *sıfırların çarpımına* gider. **Bağlantılı liste ile** her düğümde **katsayı** ve **üs** tutulur, terimler üslerin *azalan* sırasıyla dizilir:

$$P_1\text{: } [10 \mid 1000] \to [5 \mid 14] \to [1 \mid 0] \to \text{NULL} \quad (3 \text{ düğüm})$$
$$P_2\text{: } [3 \mid 1990] \to [-2 \mid 1492] \to [11 \mid 1] \to [5 \mid 0] \to \text{NULL} \quad (4 \text{ düğüm})$$

Sınav bağlantısı

Farz edin ki sorularında liste gezme kalıbı: `d = liste.head; while (d != NULL) { …d = d->next; }`. Hocanın kabul ettiği temel işlemler: `head`, `next`, `FindNode/find`, `InsertNode`, `DeleteNode`. Yılı bilinmeyen bir vizede *"bağlantılı liste ekleme akış diyagramı, karmaşıklığı"* soruldu ve fonksiyonu öğrencinin yazması istendi.